

L&T EDUTECH

MSc Artificial Intelligence – Practical Assignment

TASK 7

Advanced Multi-Page Streamlit Application

MNIST Handwritten Digit Classification using Deep Learning

Student Name:	Saikumar Padala
Roll No:	15
PRN / Student ID:	5711387
Programme:	MSc Artificial Intelligence – Part II
Dataset:	MNIST Handwritten Digit Dataset
Model:	Convolutional Neural Network (CNN)
Application:	Multi-Page Streamlit Web Application

Report Scope: This enhanced report documents the design, implementation, testing, visualization and observed results of the multi-page Streamlit application. The supplied application screenshots are embedded at the relevant sections so that the report provides direct visual evidence of the implemented interface.

Prepared for academic submission

Contents

No.	Section	Page
1	Introduction	3
2	Project Objective	3
3	Dataset Description	3
4	Application Architecture	4
5	Methodology and Processing Workflow	4
6	Technologies Used	5
7	Model Architecture	5
8	Model Performance Page	6
9	Sample Prediction Analysis	7
10	Data Visualization	8
11	Explore Digit Classes	9
12	MNIST Sample Images	10
13	Project Report Page	11
14	Testing and Validation	12
15	Observations	13
16	Conclusion	13
17	Project File Structure	14
18	Execution Instructions	14
19	Future Scope and Enhancement	15
20	Final Screenshot Placement Checklist	15

1. Introduction

This task develops a professional multi-page Streamlit application for MNIST handwritten digit classification using a trained Convolutional Neural Network (CNN). The application converts a deep-learning model into an interactive user interface where prediction, evaluation, visualization and documentation are available through separate application sections. This structure demonstrates how a machine-learning model can be presented as a usable application rather than only as a training notebook.

The application is designed around a simple user journey: a user can understand the project from the dashboard or report, inspect the model's measured performance, explore the MNIST dataset, and review classification outputs. The screenshots supplied with this submission show the implemented Streamlit pages, including model metrics, sample analysis, dataset distribution, digit exploration and the embedded report.

2. Project Objective

- Develop an advanced multi-page Streamlit application with clear navigation and a consistent presentation structure.
- Integrate a trained CNN model capable of classifying handwritten MNIST digits from 0 to 9.
- Provide an easy-to-use digit prediction interface that accepts common image formats and presents a predicted class with confidence information.
- Display model evaluation metrics, including test accuracy, test loss and the number of test samples used for evaluation.
- Provide interactive dataset visualizations such as class distribution and representative handwritten samples.
- Include an in-application project report so that methodology, results and technical details can be reviewed without leaving the application.
- Demonstrate practical integration of deep learning inference, data visualization and web-based reporting in one application.

3. Dataset Description

The MNIST dataset contains grayscale images of handwritten digits representing the ten classes from 0 through 9. Each image is represented at a resolution of 28×28 pixels. The standard dataset split used in the project contains 60,000 training images and 10,000 testing images. The dataset is therefore suitable for demonstrating multi-class image classification because each input has a compact and standardized representation.

Attribute	Description
Dataset	MNIST Handwritten Digit Dataset
Training images	60,000
Testing images	10,000
Image size	28×28 pixels
Image type	Grayscale
Attribute	Description
Classes	10 (0–9)
Task	Image Classification

4. Application Architecture

The application follows a modular multi-page architecture. The main Streamlit interface provides navigation, while individual page modules separate prediction, performance analysis, visualization and reporting responsibilities. This organization improves maintainability because each page has a focused purpose and can be updated without redesigning the entire interface.

Pipeline: Streamlit interface → image upload → grayscale conversion → resize to 28×28 → normalization → CNN inference → predicted digit and confidence → dashboards and reports.

Component	Purpose
Dashboard / Home	Project overview, system status, model information and workflow.
Digit Prediction	Upload an image and obtain the predicted digit and confidence.
Model Performance	Display test accuracy, test loss, test samples and model architecture.
Data Visualization	Explore class distribution and representative handwritten digit samples.
Project Report	Present objective, methodology, technologies, results, observations and conclusion.

5. Methodology and Processing Workflow

The prediction workflow begins when an image is supplied to the application. The input is converted to grayscale to match the MNIST image representation, resized to 28×28 pixels, and normalized before it is passed to the trained CNN. The model produces class probabilities for the ten digit classes, and the class with the highest predicted probability is presented as the final digit. This preprocessing sequence keeps the inference input consistent with the representation expected by the trained model.

In addition to inference, the application exposes evaluation and visualization functions. The performance page reports the observed test metrics and shows the network layers, while the visualization page makes the dataset easier to inspect through charts and image grids. The project-report page then consolidates the technical explanation into the same web application.

6. Technologies Used

- **Python:** Primary programming language used for application logic and model interaction.
- **Streamlit:** Framework used to construct the multi-page web interface, navigation and dashboard-style presentation.
- **TensorFlow / Keras:** Used for loading and performing inference with the trained CNN model.
- **NumPy:** Used for numerical operations and image-array processing during preprocessing and prediction.
- **Pillow:** Used for opening uploaded images, grayscale conversion and resizing.
- **MNIST:** Handwritten digit dataset used for training/testing and interactive visualization.

7. Model Architecture

The CNN used in the application contains two convolutional layers followed by max-pooling operations. The extracted feature maps are flattened into a one-dimensional representation and passed to a 64-unit dense layer. The final dense layer contains 10 output units, corresponding to the ten MNIST digit classes. This architecture provides a compact end-to-end classification pipeline in which convolutional layers learn spatial features and dense layers perform the final class decision.

The architecture shown in the application is also useful for verification because the output shapes and parameter counts make the model configuration explicit. The largest parameter contribution in the displayed architecture comes from the 64-unit dense layer, while the final 10-unit layer maps the learned representation to the digit classes.

Layer	Type	Output Shape	Parameters
conv2d	Conv2D	(None, 26, 26, 32)	320
max_pooling2d	MaxPooling2D	(None, 13, 13, 32)	0
conv2d_1	Conv2D	(None, 11, 11, 64)	18,496
max_pooling2d_1	MaxPooling2D	(None, 5, 5, 64)	0
flatten	Flatten	(None, 1600)	0
dense	Dense	(None, 64)	102,464
dense_1	Dense	(None, 10)	650

The parameter counts and tensor dimensions reproduced above are taken from the supplied project report and correspond to the architecture displayed by the Streamlit Model Performance page.

8. Model Performance Page

The Model Performance page provides a compact evaluation dashboard for the trained CNN. The supplied screenshot shows a test accuracy of 98.97%, a test loss of 0.0288 and 10,000 test samples. These values provide a direct summary of the model's observed performance on the test set used by the application. The same page also presents the model architecture, allowing the user to relate the numerical results to the implemented network structure.

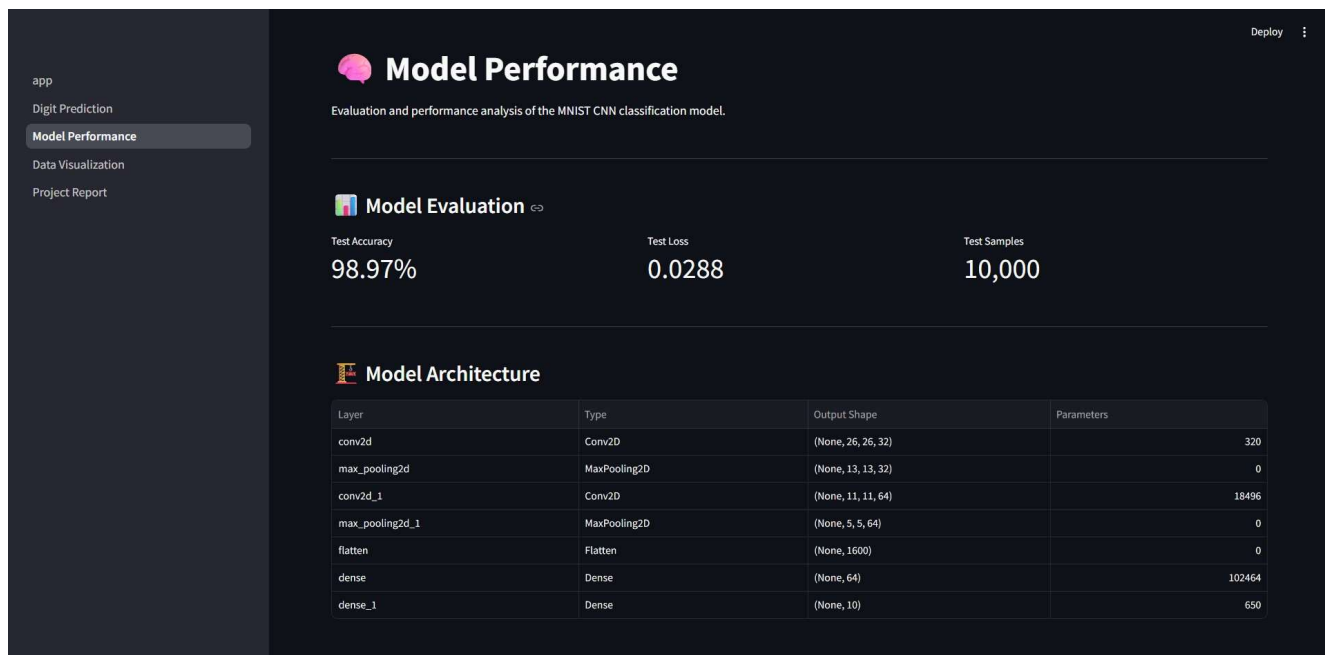


Figure 1: Streamlit Model Performance page showing evaluation metrics and CNN architecture.

The visual design separates evaluation metrics from architecture details so that a reviewer can identify the headline results quickly and then inspect the layer-by-layer configuration. This is particularly useful in an academic demonstration because it connects model implementation details with measured results in the same interface.

9. Sample Prediction Analysis

The sample prediction analysis compares actual labels with predicted labels for individual test samples. The supplied screenshot demonstrates that the application exposes the sample index, actual digit, predicted digit and a correctness indicator. The displayed summary reports 19 correct predictions among the first 20 samples, which provides a small, human-readable validation view in addition to the aggregate test accuracy.

app

Digit Prediction

Model Performance

Data Visualization

Project Report

Sample Prediction Analysis

	Sample	Actual Digit	Predicted Digit	Correct
0	1	7	7	☒
1	2	2	2	☒
2	3	1	1	☒
3	4	0	0	☒
4	5	4	4	☒
5	6	1	1	☒
6	7	4	4	☒
7	8	9	9	☒
8	9	5	5	☒
9	10	9	9	☒
				☒

Correct predictions among first 20 samples: 19/20

Figure 2: Sample Prediction Analysis showing actual versus predicted digits.

This section is valuable because aggregate metrics alone do not show individual prediction behavior. By comparing actual and predicted classes row by row, the user can inspect examples of successful classification and identify any visible mismatches in the sampled records.

10. Data Visualization

The Data Visualization page provides an overview of the MNIST dataset before users explore individual handwritten samples. The supplied interface reports 60,000 training images, 10,000 testing images, an image size of 28×28 pixels and 10 classes. It also presents a bar chart for the distribution of digit classes, making it easier to compare the number of examples available for each label.

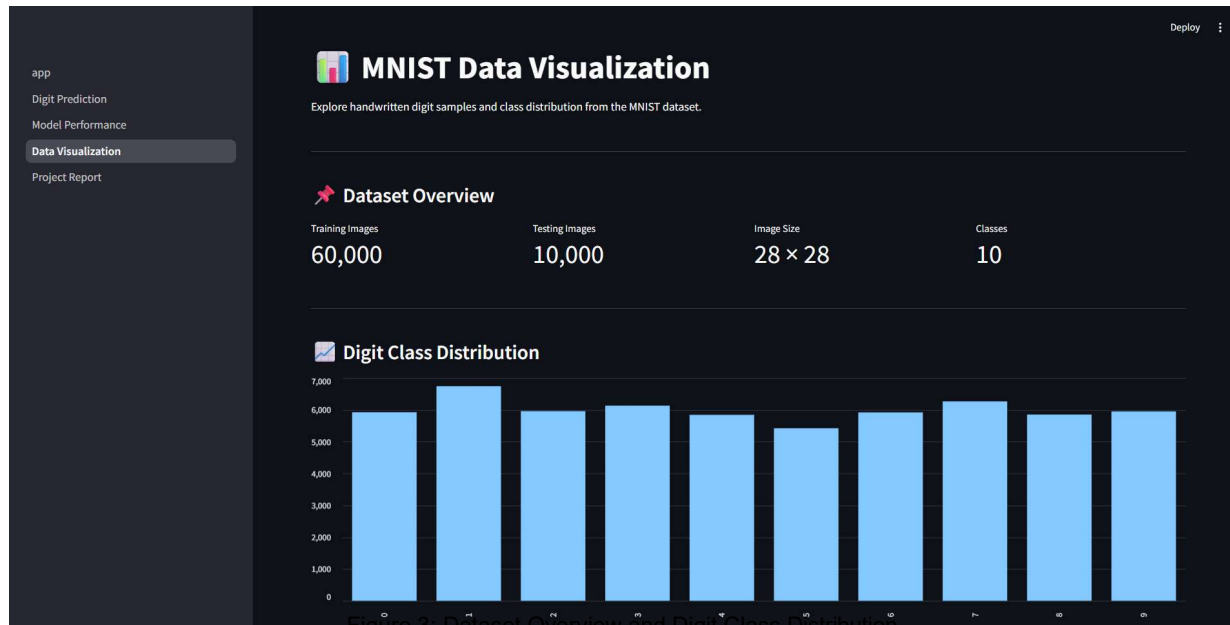


Figure 3: Dataset Overview and Digit Class Distribution.

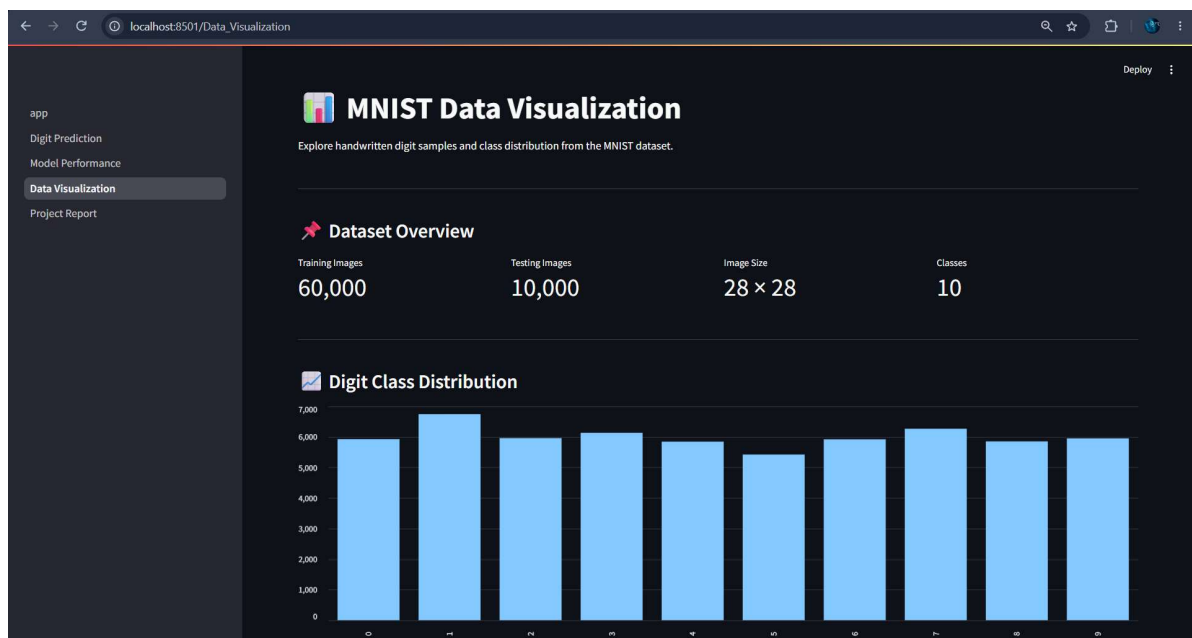


Figure 4: Alternate captured view of the MNIST dataset overview and class-distribution chart.

11. Explore Digit Classes

The class explorer allows the user to select a digit and inspect representative handwritten examples from that class. The supplied screenshots show digit 2 selected, with multiple visual variations displayed in a grid. These examples illustrate that the same class can contain substantial differences in handwriting style, stroke thickness, curvature and overall shape.

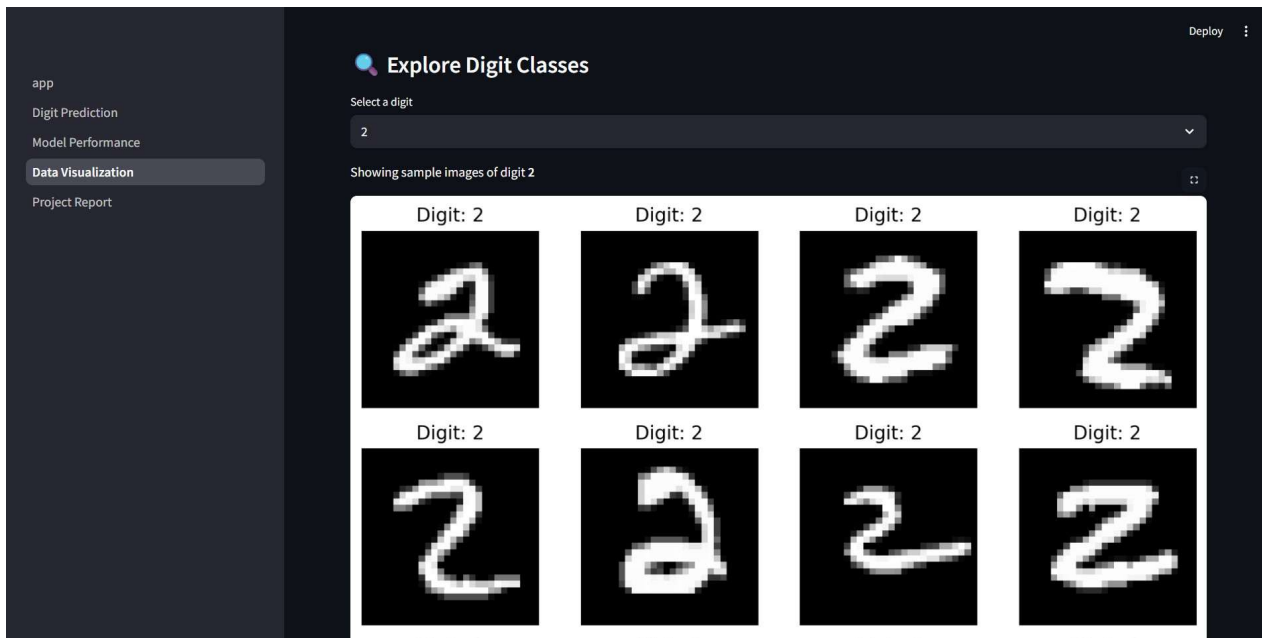


Figure 5: Interactive Explore Digit Classes page with digit 2 selected.

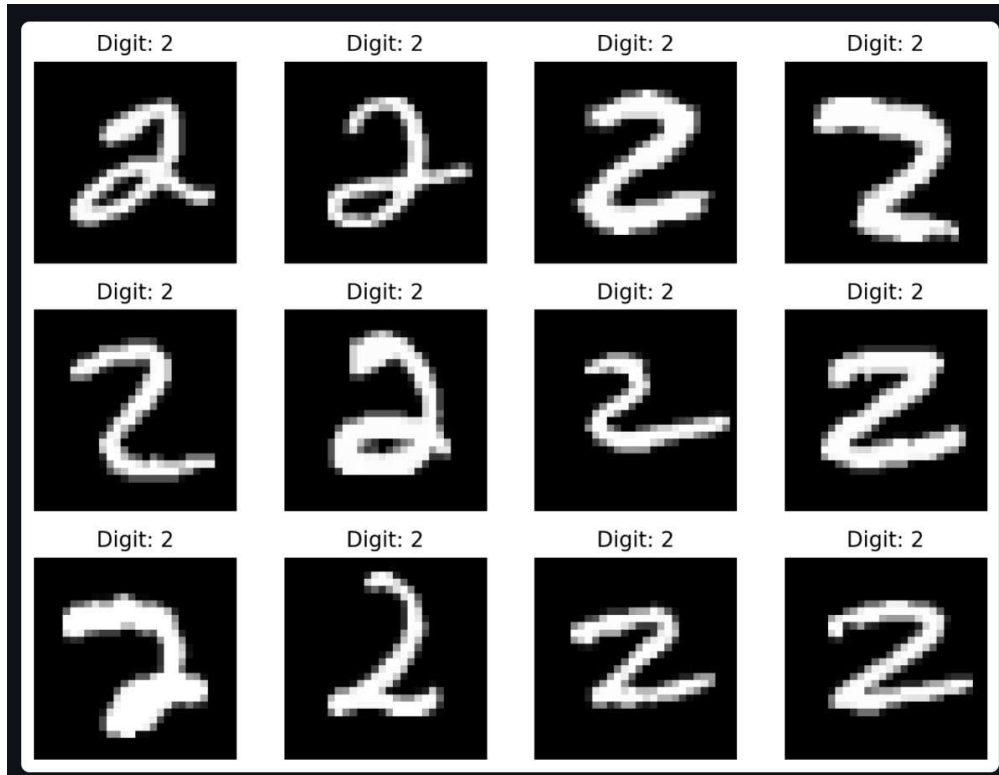


Figure 6: Enlarged MNIST digit-class sample grid captured from the application.

This interactive view supports visual dataset inspection and helps explain why image classification requires feature learning rather than relying on a single fixed representation of each digit. The examples also provide a useful demonstration of how Streamlit can combine model-oriented functionality with exploratory data analysis.

12. MNIST Sample Images

The MNIST Sample Images section displays handwritten examples together with their labels. The supplied screenshots show different classes such as 1, 2, 4, 5, 6, 7 and 8. Presenting labeled images makes the dataset more intuitive for a reviewer and provides visual evidence that the application is working with the intended handwritten-digit data rather than arbitrary images.

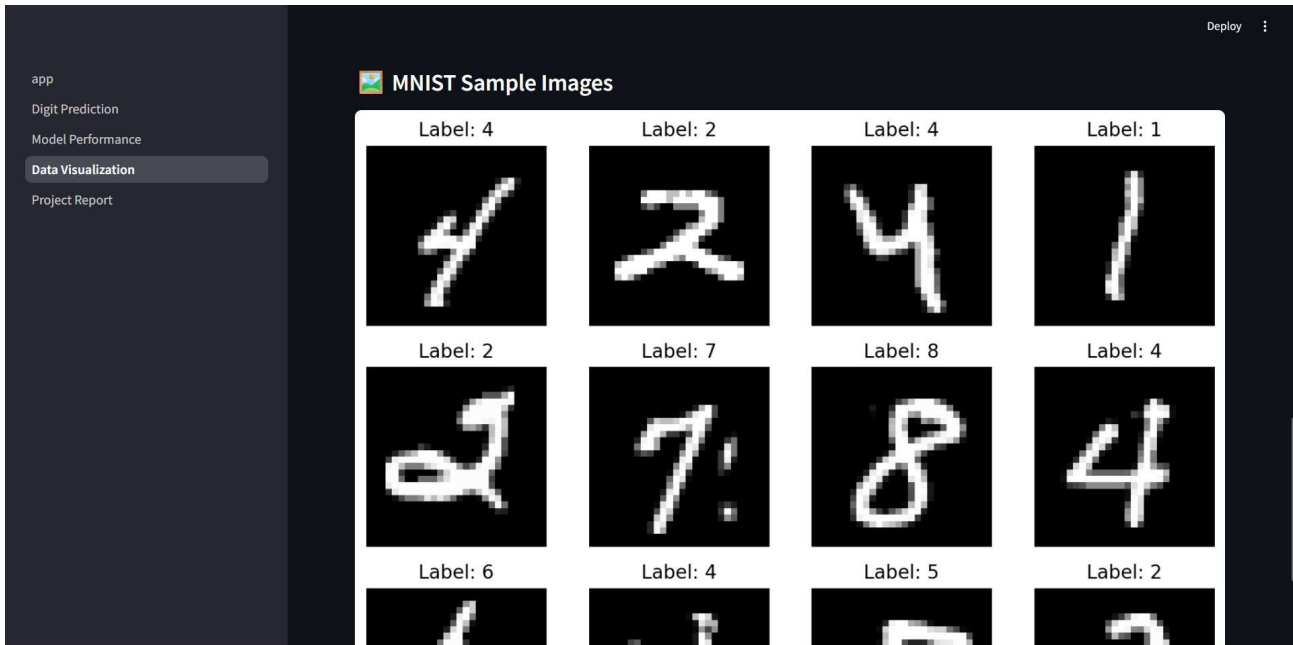


Figure 7: MNIST Sample Images page showing labeled handwritten examples.

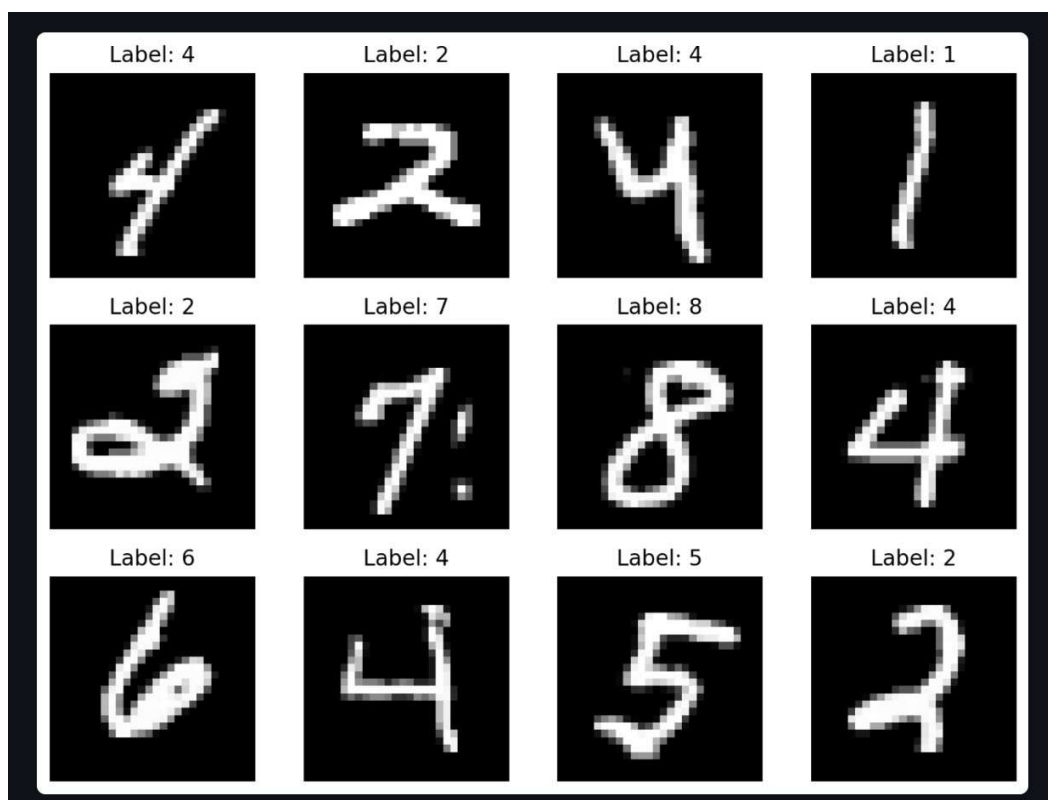


Figure 8: Enlarged labeled MNIST sample grid.

13. Project Report Page

The Project Report page embeds the project's documentation directly inside the Streamlit application. The supplied screenshot shows the report beginning with the project objective and dataset description. This design makes the application self-documenting: a reviewer can understand the purpose and data characteristics from the same interface used to demonstrate the model.

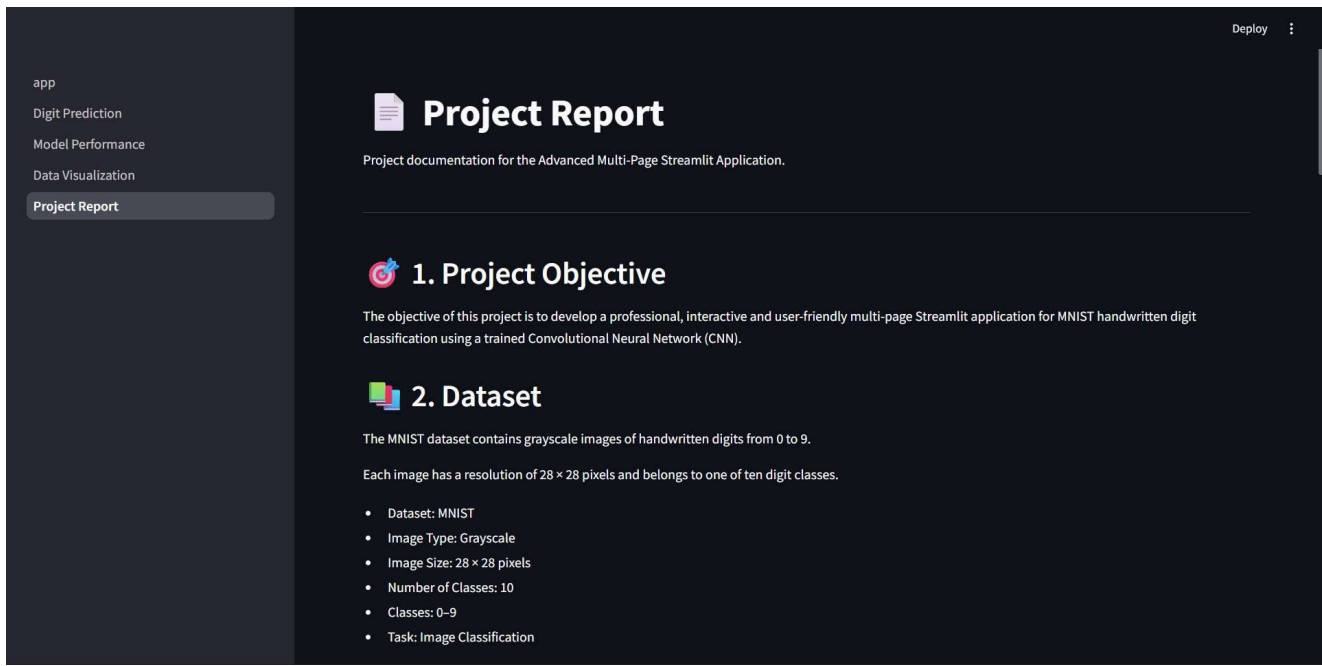


Figure 9: Embedded Project Report page in the multi-page Streamlit application.

Including documentation as an application page also improves presentation quality because the technical explanation, dataset details and implementation context remain connected to the interactive demonstration. It can therefore serve as a compact reference page during project evaluation or presentation.

14. Testing and Validation

The application was tested using the handwritten digit image named 7.png. According to the supplied report, the application displayed digit 7 with 99.99% confidence. The Model Performance page separately reports an overall test accuracy of 98.97%, a test loss of 0.0288 and 10,000 test samples. The first-20 sample analysis reports 19 correct predictions.

Metric / Test	Observed Result
Test Accuracy	98.97%
Test Loss	0.0288
Test Samples	10,000
Uploaded Test Image	7.png
Predicted Digit	7
Displayed Confidence	99.99%
First 20 Sample Predictions	19/20 correct

These observations provide two complementary forms of validation: aggregate evaluation on the test set and direct inspection of selected predictions. The uploaded-image result demonstrates interactive inference, while the test metrics and sample analysis provide evidence for evaluating the trained model within the application.

15. Observations

- The multi-page architecture separates prediction, evaluation, visualization and documentation into clearly identifiable sections.
- The trained CNN loads successfully and provides interactive inference through the Streamlit interface.
- The model performance page makes accuracy, loss, test-set size and architecture information available in one location.
- The MNIST class distribution is visualized through a bar chart, making differences between class counts easier to inspect.
- The class explorer allows users to select a digit and inspect multiple handwritten examples.
- The sample analysis page exposes individual actual and predicted labels rather than relying only on aggregate accuracy.
- The project-report page provides an integrated documentation layer, reducing the need to switch between the application and an external report.
- The dashboard-style organization improves navigation and presentation compared with a single-page prediction interface.

16. Conclusion

The Task 7 implementation demonstrates an advanced Streamlit application for MNIST handwritten digit classification. It extends a basic model-inference workflow into a multi-page application containing model evaluation, dataset visualization, interactive class exploration and embedded project documentation. The supplied screenshots show that the principal application sections are functioning as intended and present the results in a structured format.

The implementation addresses the major requirements of the task: multi-page architecture, CNN integration, dashboard-style visualization, performance metrics, interactive charts, sample analysis, reporting and improved user experience. The resulting application therefore demonstrates both the machine-learning component and the practical software-interface component required to present a trained deep-learning model to users.

17. Project File Structure

The project is organized so that the main Streamlit entry point, trained model and dependency specification are kept at the project root, while individual application pages are maintained inside the pages directory.

This separation supports Streamlit's multi-page convention and makes the codebase easier to navigate and maintain.

Task_7_Advanced_MultiPage_Streamlit/
app.py
deep_learning_model.h5
requirements.txt
pages/
1_Digit_Prediction.py
2_Model_Performance.py
3_Data_Visualization.py
4_Project_Report.py

18. Execution Instructions

To execute the application, the four Python page files should remain inside the pages folder so that Streamlit can discover the individual application sections. Required Python packages should be installed from the requirements file, after which the application can be launched from the project root.

- Keep the four Python page files inside the pages folder.
- Install dependencies using **pip install -r requirements.txt**.
- Run the application using **streamlit run app.py**.
- Open the Streamlit address, normally **http://localhost:8501**.
- Use the sidebar to navigate through the available application sections.

19. Future Scope and Enhancement

The current application provides the core functionality required for MNIST classification and interactive reporting. A natural next step would be to extend the prediction interface with additional input methods, such as a drawing canvas, while retaining the same preprocessing pipeline. This would allow a user to draw a digit directly in the browser and send the resulting image to the CNN for inference.

Further enhancement could include richer evaluation visualizations such as a confusion matrix, per-class accuracy analysis, confidence thresholds and downloadable prediction reports. The application could also expose model metadata, training-history plots and additional validation examples. These enhancements would build on the existing multi-page structure without changing the core purpose of the project.

For deployment, the application can be packaged with its model file and dependency specification and then hosted on an appropriate Streamlit-compatible environment. Additional input validation, exception handling and resource caching could also be added to improve robustness when multiple users interact with the application.

20. Final Screenshot Placement Checklist

Figure	Screenshot / Evidence
1	Model Performance page – evaluation metrics and CNN architecture
2	Sample Prediction Analysis
3	Dataset Overview and Digit Class Distribution
4	Alternate Dataset Overview capture
5	Explore Digit Classes
6	Enlarged digit-class sample grid
7	MNIST Sample Images
8	Enlarged labeled MNIST sample grid
9	Project Report page

The enhanced report embeds the uploaded screenshots directly under their relevant sections. All report text is rendered in black to provide a consistent print-friendly appearance, while the screenshots retain their original application appearance so that the evidence remains faithful to the supplied implementation.

GitHub link for LMS

<https://github.com/Saipadala2003/Task-7-Advanced-MultiPage-Streamlit-MNIST.git>